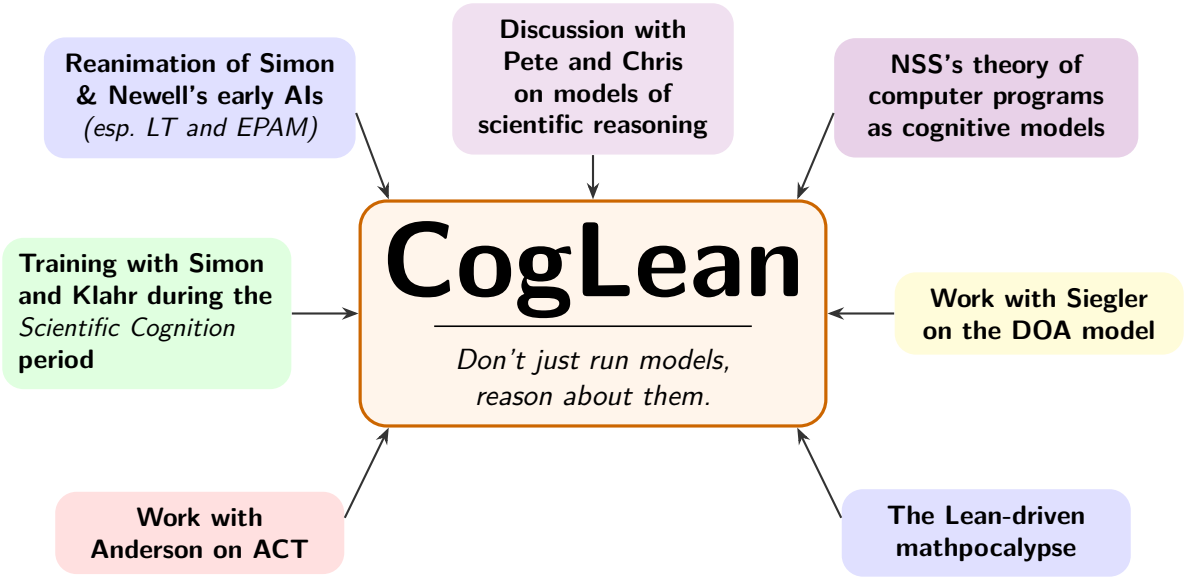
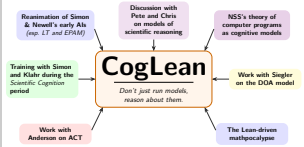
Epoofany

Epoofany

Epoofany.


Reanimation of Simon & Newell's early AIs (esp. LT and EPAM)
Discussion with Pete and Chris on models of scientific reasoning
NSS's theory of computer programs as cognitive models
Training with Simon and Klahr during the Scientific Cognition period
CogLean
Don't just run models, reason about them.
Work with Siegler on the DOA model
Work with Anderson on ACT
The Lean-driven mathpocalypse

CogLean



Where CogLean came from.

CogLean

Towards a Cumulative, Machine-Checked Theory of Cognition

Jeff Shrager

November 2026

Draft for Comment

CogLean



This talk goes with the paper of the same name, which is a draft for comment. It hasn't been carefully checked yet: think of it as an over-starched blog post, and constructive feedback is welcome. It introduces CogLean, a library of cognitive theory in the Lean proof assistant, and argues for a way of doing theoretical cognitive science: theories stated once, their consequences proved, and results that accumulate, compose and transfer. I'm going to tell it partly through a hypothetical modeler, Chris, who uses the library to answer two questions: one forward, from a phenomenon to a theorem, and one backward, from a proposed regularity to what a theory must be like to produce it. Companion talks and papers take it further, to a model of multi-digit addition and to concept formation. And one thing up front: this is less about proof, in the usual sense, than about explanation. The theorems are explanations of phenomena, and Lean keeps them aligned with the programs they explain.

The problem: making theory cumulative

- Newell (1973): twenty questions with nature, answers never assembled; Newell (1990): unified theories.
- Cognitive architectures (Soar, ACT-R; now the Common Model) have spent decades on this: **shared mechanisms**, so models of different tasks reuse and integrate theoretical commitments.
- What they accumulate is mainly **models**: what a model predicts is established by running it, one setting at a time, and a result for one model is rarely reusable *as a result* by another.
- CogLean, **complementary**: accumulate machine-checked **theorems** about mechanisms, reusable deductively, and stated across architectural commitments.

Can cognitive theory be organized the way Mathlib organizes mathematics: definitions made once, results proved, and later work building on earlier results?

The problem: making theory cumulative

- Newell (1973): twenty questions with nature, answers never assembled; Newell (1990): unified theories.
- Cognitive architectures (Soar, ACT-R; now the Common Model) have spent decades on this: **shared mechanisms**, so models of different tasks reuse and integrate theoretical commitments.
- What they accumulate is mainly **models**: what a model predicts is established by running it, one setting at a time, and a result for one model is rarely reusable as *a result* by another.
- CogLean, **complementary**: accumulate machine-checked **theorems** about mechanisms, reusable deductively, and stated across architectural commitments.

Can cognitive theory be organized the way Mathlib organizes mathematics: definitions made once, results proved, and later work building on earlier results?

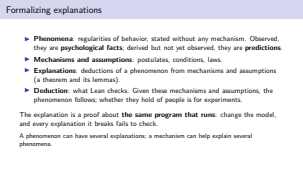
The problem is an old one, and the people who have done the most about it are the architecture builders. Newell complained in 1973 that we play twenty questions with nature and never assemble the answers, and his answer was unified theories of cognition. Soar and ACT-R, and now the Common Model of Cognition, are decades of work by Newell, Anderson, Laird, Rosenbloom and many others on exactly this: shared mechanisms and constraints, so that a model of a new task reuses the architecture's theory of memory, learning and control, and models of different tasks can be integrated. CogLean is not the first effort in this direction, and it isn't a competitor. What architectures accumulate, though, is mainly models. What a model predicts is established by running it, at one setting at a time, and a result established for one model is rarely available as a result to another. CogLean pursues a complementary goal: accumulating machine-checked theorems about cognitive mechanisms, which can be reused deductively, and stated independently of any one architecture's commitments, so they can cover classes of mechanisms. An architecture could, in principle, carry such results itself. The question for this talk is whether cognitive theory can be organized the way mathematics has been organized in Mathlib, so that results, and not

Formalizing explanations

- **Phenomena**: regularities of behavior, stated without any mechanism. Observed, they are **psychological facts**; derived but not yet observed, they are **predictions**.
- **Mechanisms and assumptions**: postulates, conditions, laws.
- **Explanations**: deductions of a phenomenon from mechanisms and assumptions (a theorem and its lemmas).
- **Deduction**: what Lean checks. Given these mechanisms and assumptions, the phenomenon follows; whether they hold of people is for experiments.

The explanation is a proof about **the same program that runs**: change the model, and every explanation it breaks fails to check.

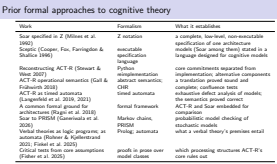
A phenomenon can have several explanations; a mechanism can help explain several phenomena.



Before any Lean, here's what this is for. It's less about proof, in the usual sense, than about explanation. What we explain are phenomena, stated without any mechanism: observed, they're psychological facts; derived but not yet observed, they're predictions. Underneath are mechanisms and their assumptions: postulates, conditions and laws. An explanation is a deduction of a phenomenon from those, in Lean a theorem and its chain of lemmas. That's Hempel and Oppenheim's old deductive picture of explanation, running through how the mechanisms work, made checkable. Lean proves conditional statements; whether the assumptions hold of people is for experiments. And here's what I think matters most. Usually a model is a program and its explanation lives somewhere else, in prose or in math about an idealized program, and the two drift apart. In Lean, the definitions that run are the ones the theorems are about, so when the model changes, every explanation it breaks fails to check, and the rest are confirmed. Not every proof is an explanation; that stays a judgment. But one phenomenon can have several explanations, and one mechanism can help explain several phenomena, and a library of them lets us ask what observations would tell explanations apart.

Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of one architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Frühwirth 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar to PRISM (Ganeriwala et al. 2026)	Markov chains, PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rohner & Kjellerstrand 2021; Finkel et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2025)	proofs in prose over model classes	which processing structures ACT-R's core rules out


Prior formal approaches to cognitive theory

Work	Formalism	What it establishes
Soar specified in Z (Milnes et al. 1992)	Z notation	a complete, low-level, non-executable specification of core architecture
Sceptic (Cooper, Fox, Farringdon & Shallice 1996)	executable specification language	models (Soar among them) stated in a language designed for cognitive models
Reconstructing ACT-R (Stewart & West 2007)	Python reimplementation	core commitments separated from implementation; alternative components
ACT-R operational semantics (Gall & Fridsma 2018)	abstract semantics; CHR	a translation proved sound and complete; confluence tests
ACT-R as timed automata (Langenfeld et al. 2019, 2021)	timed automata	exhaustive defect analysis of models; the semantics proved correct
A common formal ground for architectures (Ragni et al. 2018)	formal framework	ACT-R and Soar embedded for comparison
Soar in PRISM (Gluck & Laird 2018)	Markov chains; PRISM	probabilistic model checking of stochastic models
Verbal theories as logic programs; as automata (Rubow & Kjellerstrand 2021; Fisher et al. 2025)	Prolog; automata	what a verbal theory's premises entail
Critical tests from core assumptions (Fisher et al. 2026)	proofs in prose over model classes	which processing structures ACT-R's core rules out

CogLean is far from the first formal treatment of cognitive theory, and it's worth seeing what came before. In 1992 Milnes and colleagues specified Soar in the Z notation: complete, but low-level and not executable. Cooper, Fox, Farringdon and Shallice argued in 1996 for languages designed for specifying cognitive models, and built one, Sceptic. Stewart and West rebuilt ACT-R to separate its core commitments from its implementation. For ACT-R there's a formal operational semantics with a translation proved sound and complete, and a line of work translating ACT-R models into timed automata for exhaustive defect checking. Ragni and colleagues proposed a common formal ground into which ACT-R and Soar can be embedded. Most recently, Soar models have been compiled for the PRISM probabilistic model checker. Outside architectures, verbal theories have been formalized as logic programs and as automata, and Fisher and colleagues derived critical tests of ACT-R from its core assumptions. Almost all of this work formalizes one architecture or one model at a time, and establishes properties of that system. That's the contrast I'll come back to.

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.

The analogy: Mathlib

- Mathlib, the community library of formalized mathematics in Lean, proposes no new mathematics.
- It is a **shared environment**: definitions made once, every result machine-checked, later work citing earlier results.
- A theorem about groups, once proved, is available to everyone who uses groups.

CogLean asks whether cognitive theory can be organized the same way.
It is not a new theory of cognition, nor a new modeling language.
And it is nowhere near Mathlib's scale: a first demonstration.

Mathematics faced a version of this problem and has made real progress on it. Mathlib, the community library for the Lean proof assistant, doesn't propose new mathematics. It's a shared environment: definitions are made once, every result is checked by machine, and later work builds on earlier results by citing them. A theorem about groups, once proved, is available to everyone who works with groups. CogLean asks whether cognitive theory can be organized the same way. It is not a new theory of cognition, and it's not a new modelling language either. And it's nowhere near Mathlib's scale; this is a first demonstration that the approach can work.

CogLean's layers

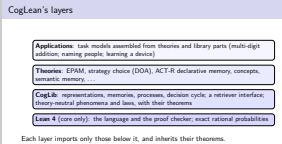
Applications: task models assembled from theories and library parts (multi-digit addition; naming people; learning a device)

Theories: EPAM, strategy choice (DOA), ACT-R declarative memory, concepts, semantic memory, ...

CogLib: representations, memories, processes, decision cycle; a retriever interface; theory-neutral phenomena and laws, with their theorems

Lean 4 (core only): the language and the proof checker; exact rational probabilities

Each layer imports only those below it, and inherits their theorems.



CogLean has four layers. At the bottom is Lean, core only, with exact rational probabilities. Above it is CogLib, the general library: representations such as feature codes and distributions; memory structures such as discrimination nets and an activation memory; processes such as sampled retrieval and a decision cycle with impasses; an interface that any memory can implement; and theory-neutral statements of phenomena and laws, with theorems about them. Above that are the theories, each stated as choices of library mechanisms. And at the top are applications, models of particular tasks assembled from theories and library parts. Each layer imports only those below it, and inherits their theorems.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("x is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Four kinds of object have to be kept apart. Mechanisms and their postulates are definitions. Phenomena are predicates, stated without reference to any mechanism; a predicate asserts nothing, it's a target. Theorems say that a mechanism produces behaviour satisfying a predicate. And conditions are the hypotheses under which a theorem holds: some situational, some empirical premises we haven't explained yet, some just well-formedness. One rule follows: a theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction of any model. The library's catalog flags those mechanically.

Four kinds of object, kept apart

1. **Mechanisms** and their postulates: definitions (how a net sorts a code; how retrieval samples).
2. **Phenomena**: predicates, stated without any mechanism. A predicate asserts nothing; it is a target.
3. **Theorems**: a mechanism produces behavior satisfying a predicate.
4. **Conditions**: situational ("r is confused with q"), empirical premises not yet explained ("associations are sharp"), well-formedness.

A theorem with a phenomenon among its hypotheses is a relation between phenomena, not a prediction. The catalog flags these mechanically.

Scientific notions, and what they are in Lean

Model (program)	executable definitions: the same ones that run
Mechanism	a CogLib process, chosen by a theory
Phenomenon	a predicate, with no mechanism: a target
Psychological fact	a phenomenon judged to hold of people: *outside Lean*
Prediction	a theorem's conclusion, not yet observed
Condition	a theorem's hypothesis
Law	a property of a mechanism: proved for a family, or required by a class theorem
Explanation	a theorem and its lemmas
Deduction	what Lean's kernel checks
Illustration	a value computed at one setting: a witness, not an explanation
Lean's axioms	logic only, never scientific assumptions

Scientific notions, and what they are in Lean

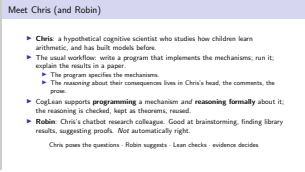
Model (program)	executable definitions: the same ones that run
Mechanism	a CogLib process, chosen by a theory
Phenomenon	a predicate, with no mechanism: a target
Psychological fact	a phenomenon judged to hold of people: outside *Lean*
Prediction	a theorem's conclusion, not yet observed
Condition	a theorem's hypothesis
Law	a property of a mechanism: proved for a family, or required by a class theorem
Explanation	a theorem and its lemmas
Deduction	what Lean's kernel checks
Illustration	a value computed at one setting: a witness, not an explanation
Lean's axioms	logic only, never scientific assumptions

To be completely explicit about how the words line up. The model, the program, is a set of executable definitions, and they're the same definitions that get run. A mechanism is a CogLib process that a theory chooses. A phenomenon is a predicate stated with no mechanism in it; it's a target. A psychological fact is a phenomenon that holds of people, and that judgment is outside Lean altogether. A prediction is a theorem's conclusion about something nobody has observed yet. Conditions are a theorem's hypotheses. A law is a property of a mechanism, either proved for a whole family or required by a theorem about a class. None of these are Lean axioms; a law of memory is a hypothesis in the same way the group axioms are hypotheses in Mathlib. An explanation is a theorem and its chain of lemmas, and deduction is what Lean's kernel checks. A number computed at one setting is an illustration, a witness that the hypotheses can be met, not an explanation. And Lean's own three axioms are just logic.

Meet Chris (and Robin)

- **Chris**: a hypothetical cognitive scientist who studies how children learn arithmetic, and has built models before.
- The usual workflow: write a program that implements the mechanisms; run it; explain the results in a paper.
  - The program specifies the mechanisms.
  - The *reasoning* about their consequences lives in Chris's head, the comments, the prose.
- CogLean supports **programming** a mechanism *and* **reasoning formally** about it; the reasoning is checked, kept as theorems, reused.
- **Robin**: Chris's chatbot research colleague. Good at brainstorming, finding library results, suggesting proofs. *Not* automatically right.

Chris poses the questions · Robin suggests · Lean checks · evidence decides



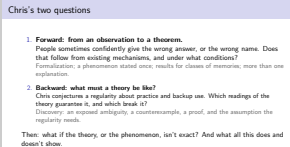
Let me introduce Chris. Chris is a hypothetical cognitive scientist who studies how children learn arithmetic, and has built computational models before. Like most of us, Chris is used to a particular workflow: write a program that implements the proposed mechanisms, run it to see what it does, and explain the results in a paper. The program specifies the mechanisms, but the reasoning about what they imply lives in Chris's head, in the comments, or in the prose. CogLean offers another way to work. It supports programming a mechanism and also reasoning formally about it, and that reasoning is checked by the computer, kept as theorems, and reusable in other models. Chris also has a colleague, Robin, who is a chatbot. Robin is genuinely useful: for brainstorming, for finding what's already in the library, for suggesting proofs. But Robin is not automatically right, and you'll see Robin be wrong. The division of labor is: Chris poses the questions, Robin suggests, Lean checks, and evidence decides which premises describe children.

Chris's two questions

1. **Forward: from an observation to a theorem.**
   People sometimes confidently give the wrong answer, or the wrong name. Does that follow from existing mechanisms, and under what conditions?
   Formalization; a phenomenon stated once; results for classes of memories; more than one explanation.

2. **Backward: what must a theory be like?**
   Chris conjectures a regularity about practice and backup use. Which readings of the theory guarantee it, and which break it?
   Discovery: an exposed ambiguity, a counterexample, a proof, and the assumption the regularity needs.

Then: what if the theory, or the phenomenon, isn't exact? And what all this does and doesn't show.



Chris has two questions, and they're the first two parts of the talk. The first goes forward, from an observation to a theorem. People sometimes confidently give a wrong answer, a child saying two hundred to one fifty plus one hundred, or any of us greeting a stranger by the wrong name. Does that follow from mechanisms we already have, and under exactly what conditions? The second goes backward. Chris conjectures a regularity about practice and backup use, and asks what a theory has to be like to guarantee it. That one exposes an ambiguity in the theory, produces a counterexample and a proof, and isolates the assumption the regularity needs. After that, a part on what happens when the theory or the phenomenon isn't exact, with the power law of practice as the case, and then what all this does and doesn't show.

Part 1

From an observation to a theorem

Does confident error follow from mechanisms we already have?

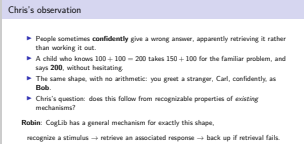
Part one. Chris's first question, going forward: does a familiar phenomenon, confident errors, follow from mechanisms already in the library?

Chris's observation

- People sometimes **confidently** give a wrong answer, apparently retrieving it rather than working it out.
- A child who knows $100 + 100 = 200$ takes $150 + 100$ for the familiar problem, and says **200**, without hesitating.
- The same shape, with no arithmetic: you greet a stranger, Carl, confidently, as **Bob**.
- Chris's question: does this follow from recognizable properties of *existing* mechanisms?

Robin: CogLib has a general mechanism for exactly this shape,

recognize a stimulus → retrieve an associated response → back up if retrieval fails.



Here's Chris's observation. People sometimes give a wrong answer confidently: they seem to be retrieving it, not working it out. A child who knows one hundred plus one hundred is two hundred takes one fifty plus one hundred for the familiar problem, and says two hundred without hesitating. And the same thing happens with no arithmetic at all: you see a stranger, Carl, and confidently call him Bob. Chris's question is whether that follows from recognizable properties of mechanisms we already have, rather than from something invented for the purpose. Robin points out that the library has a general mechanism of exactly this shape: recognize a stimulus, retrieve a response associated with what was recognized, and fall back on a backup if retrieval fails. To keep things small, I'll use naming people; the companion talk does the arithmetic.

The smallest instance: naming people

- A face is a feature code. Memory recognizes it as a familiar face, or as nothing.
- The recognized person's **name associations** are searched, as in Siegler & Shrager's model: sample a name by strength, state it if it clears a criterion.
- Backup: "don't know".

EPAM (Feigenbaum & Simon): a discrimination net learns Ann's and Bob's faces. They differ in the first feature, so that's all it tests.

Carl shares Bob's first feature, so the net sorts him to Bob.

Own name .90, others .01, criterion .50.

The smallest instance: naming people

- A face is a feature code. Memory recognizes it as a familiar face, or as nothing.
- The recognized person's **name associations** are searched, as in Siegler & Shrager's model: sample a name by strength, state it if it clears a criterion.
- Backup: "don't know".

EPAM (Feigenbaum & Simon): a discrimination net learns Ann's and Bob's faces. They differ in the first feature, so that's all it tests.

Carl shares Bob's first feature, so the net sorts him to Bob.

Own name .90, others .01, criterion .50.

Here's the smallest version. A face is a code, a vector of features. Memory recognizes it as a familiar face, or as nothing. The recognized person's name associations are then searched, the way the distributions-of-associations model that Bob Siegler and I proposed searches answers: sample a name in proportion to its strength, and say it if it clears a confidence criterion. If nothing is recognized, or no name clears the criterion, the backup is don't know. For memory, take EPAM, Feigenbaum and Simon's discrimination net, formalized on its own, with its own theorems. It learns Ann's and Bob's faces, which differ in their first feature, so that's the only feature it tests. Carl, a stranger, shares Bob's first feature, so the net sorts him to Bob. Each face's own name has strength point nine, others point zero one, and the criterion is a half.

The phenomenon, stated without a mechanism, and when it follows

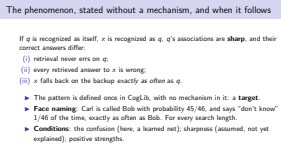
If q is recognized as itself, x is recognized as q, q's associations are **sharp**, and their correct answers differ:

(i) retrieval never errs on q;

(ii) every retrieved answer to x is wrong;

(iii) x falls back on the backup *exactly as often* as q.

- The pattern is defined once in CogLib, with no mechanism in it: a **target**.
- **Face naming**: Carl is called Bob with probability 45/46, and says "don't know" 1/46 of the time, exactly as often as Bob. For every search length.
- **Conditions**: the confusion (here, a learned net); sharpness (assumed, not yet explained); positive strengths.

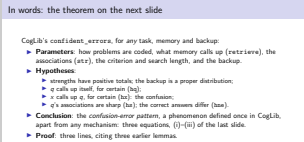


Chris and Robin first state the phenomenon without mentioning EPAM or any memory. If a familiar q is recognized as itself, a stranger x is recognized as q, q's associations are sharp, meaning only q's correct answer clears the criterion, and the correct answers differ, then: retrieval never errs on q; every retrieved answer to x is wrong; and x falls back on its backup exactly as often as q does. The pattern itself is defined once in the library, with no mechanism in it; it's a target. For face naming the conditions hold, so Carl is confidently misnamed: called Bob forty-five times in forty-six, and saying don't know exactly as rarely as Bob does, for every search length. Those numbers are witnesses that the conditions can be met. The explanation is the theorem. And the conditions are explicit: the confusion, which here comes from a learned net; sharpness, which is assumed and not yet explained; and positive strengths.

In words: the theorem on the next slide

CogLib's `confident_errors`, for *any* task, memory and backup:

- **Parameters**: how problems are coded, what memory calls up (`retrieve`), the associations (`str`), the criterion and search length, and the backup.
- **Hypotheses**:
  - strengths have positive totals; the backup is a proper distribution;
  - q calls up itself, for certain (`hq`);
  - x calls up q, for certain (`hx`): the confusion;
  - q's associations are sharp (`hs`); the correct answers differ (`hne`).
- **Conclusion**: the *confusion-error pattern*, a phenomenon defined once in CogLib, apart from any mechanism: three equations, (i)–(iii) of the last slide.
- **Proof**: three lines, citing three earlier lemmas.



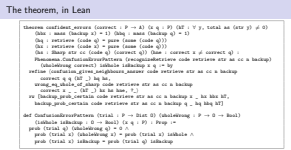
The next slide shows the actual Lean theorem, so here it is in words first. It's the library's version, which holds for any task, any memory and any backup. Its parameters are how problems are coded, what memory calls up for a code, the associations, the criterion and search length, and the backup. Its hypotheses: well-formedness of the strengths and the backup; the familiar problem calls up itself for certain; the stranger calls up the familiar problem for certain, which is the confusion; the familiar problem's associations are sharp; and the correct answers differ. The conclusion is the confusion-error pattern, which the library defines once, as a phenomenon, with no mechanism in it: the three equations from the previous slide. And the proof is three lines, each citing an earlier lemma.

The theorem, in Lean

```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    Phenomena.ConfusionErrorPattern (recognizeRetrieve code retrieve str as cc n backup)
      (wholeWrong correct) isWhole isBackup x q := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup
      correct q q (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup
      correct x _ _ (hT _) hx hs hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]

def ConfusionErrorPattern (trial : P → Dist O) (wholeWrong : P → O → Bool)
    (isWhole isBackup : O → Bool) (x q : P) : Prop :=
  prob (trial q) (wholeWrong q) = 0 ∧
    prob (trial x) (wholeWrong x) = prob (trial x) isWhole ∧
    prob (trial x) isBackup = prob (trial q) isBackup
```

Verbatim from CogLib (`Process/RecognizeRetrieve`, `Phenomena/ConfusionErrors`). Face naming and the multi-digit model are one-line instances.



Verbatim from CogLib (Process/RecognizeRetrieve, Phenomena/ConfusionErrors). Face naming and the multi-digit model are one-line instances.

Here's the theorem itself, verbatim from the library. The first five lines are the hypotheses, each named: h T, the two backup hypotheses, h q, h x, h s and h n e. Then the conclusion: the trial has the confusion-error pattern. Then the proof. At the bottom is that pattern's definition, three equations joined by and, about observable outcomes only. The first clause is the lemma that a problem recognized as a sharp q gets only q's answer; the second is the lemma that all its whole-problem answers are then wrong; and the third rewrites both backup probabilities with the lemma that backup use after certain recognition depends only on what's recognized. Nothing in the statement mentions arithmetic, EPAM or ACT-R. Face naming, and the companion talk's multi-digit model, are one-line instances of this one.

Equal is not low

- The theorem gives *equal* backup use for x and q, not *low* backup use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the stranger falls back with probability at most $(1 - r)^n$: **confident** errors, a second, quantitative phenomenon.
- With flat associations on q, the same theorem holds, and the errors are not confident.

In face naming, $R_q = 45/46$ and $n = 1$: "don't know" $1/46$ of the time.

Equal is not low

- The theorem gives *equal* backup use for x and q, not *low* backup use.
- The rate is $(1 - R_q)^n$, with R_q the chance one search of q's associations clears the criterion.
- If $R_q \geq r$, the stranger falls back with probability at most $(1 - r)^n$: **confident** errors, a second, quantitative phenomenon.
- With flat associations on q, the same theorem holds, and the errors are not confident.

In face naming, $R_q = 45/46$ and $n = 1$: "don't know" $1/46$ of the time.

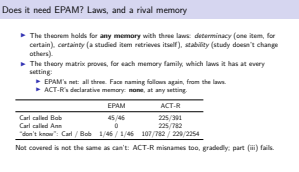
One precision matters here. The theorem says the stranger falls back on the backup exactly as often as the familiar person. It doesn't say that's rare. The rate is one minus R q to the n, where R q is the chance that one search of q's associations clears the criterion; and if R q is at least r, backup use is at most one minus r to the n. Both are proved. So whether these errors are confident, fluent and unhesitating, is a separate, quantitative phenomenon, and the library states it separately. With flat associations the same theorem holds, but the errors wouldn't be confident. In face naming R q is forty-five over forty-six, so Carl says don't know once in forty-six.

Does it need EPAM? Laws, and a rival memory

- The theorem holds for **any memory** with three laws: *determinacy* (one item, for certain), *certainty* (a studied item retrieves itself), *stability* (study doesn't change others).
- The theory matrix proves, for each memory family, which laws it has at every setting:
  - EPAM's net: all three. Face naming follows again, from the laws.
  - ACT-R's declarative memory: **none**, at any setting.

	EPAM	ACT-R
Carl called Bob	45/46	225/391
Carl called Ann	0	225/782
"don't know": Carl / Bob	1/46 / 1/46	107/782 / 229/2254

Not covered is not the same as can't: ACT-R misnames too, gradedly; part (iii) fails.



Does any of this need EPAM? No. The library proves the same pattern for any memory with three laws: determinacy, it calls up one item for certain; certainty, a studied item retrieves itself; and stability, studying one item doesn't change another's retrieval of itself. The theory matrix proves, for each memory family and each law, whether the family has it at every setting. EPAM's net has all three, so face naming follows again, this time from the laws. ACT-R's declarative memory has none of them, at any setting. But be careful about what that means. It doesn't mean ACT-R can't misname. With ACT-R's memory, Carl is called Bob more than half the time, but he can also be called Ann, and he says don't know more often than Bob does, so the third part of the pattern fails. Not covered by the theorem isn't the same as can't produce something similar.

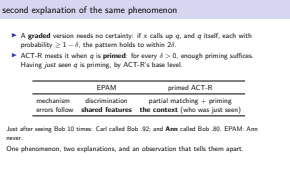
A second explanation of the same phenomenon

- A **graded** version needs no certainty: if x calls up q, and q itself, each with probability $\geq 1 - \delta$, the pattern holds to within 2δ.
- ACT-R meets it when q is **primed**: for every $\delta > 0$, enough priming suffices. Having *just seen* q is priming, by ACT-R's base level.

	EPAM	primed ACT-R
mechanism	discrimination	partial matching + priming
errors follow	**shared features**	**the context** (who was just seen)

Just after seeing Bob 10 times: Carl called Bob .92; and **Ann** called Bob .80. EPAM: Ann never.

One phenomenon, two explanations, and an observation that tells them apart.



And there's more. The library also proves a graded version of the theorem that doesn't need certainty: if the stranger calls up the familiar item, and the familiar item calls up itself, each with probability at least one minus delta, the pattern holds to within two delta. ACT-R's memory meets that condition when the context primes the familiar item, because spreading activation multiplies its weight, and for every delta, enough priming suffices. So confident misnaming now has two explanations in the library, from different mechanisms: discrimination, and partial matching with priming. And they make different predictions. EPAM's errors follow the features a stranger shares with someone you know; primed ACT-R's follow the context, like who you just saw. And what primes isn't just assumed: having just seen Bob is priming, through ACT-R's own base level. After seeing Bob ten times, Carl is called Bob ninety-two percent of the time. But so is Ann, a familiar face, eighty percent of the time; EPAM never calls her Bob. That's the observation that separates the explanations.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new task **without re-running** anything (names here; sums in the companion talk), and to *any* memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that people have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; *confident* needs a further, quantitative condition.

What Chris has now

- Not a simulation of one error, but a **reusable result**: sufficient conditions for the phenomenon, for every setting that meets them.
- It applies to a new task **without re-running** anything (names here; sums in the companion talk), and to any memory with three laws.
- The conditions are where the science is:
  - Lean hasn't shown that people have sharp associations, or that a given learning history produces the confusion: separate theoretical or empirical questions;
  - the pattern by itself doesn't make backup use rare; confident needs a further, quantitative condition.

So what does Chris have now? Not a simulation that happened to produce one error, but a reusable result: a set of conditions sufficient for the phenomenon, proved for every setting that meets them. It applies to a new task without re-running anything, and through the laws it applies to any memory with those three properties. And the conditions are where the science is. Lean hasn't shown that people have sharp associations, or that a particular learning history produces the confusion; those are separate questions, theoretical or empirical. And the pattern alone doesn't make backup use rare: calling the errors confident takes the further quantitative condition we just saw.

Part 2

Working backward

What must a theory be like to guarantee a regularity?

Part two. Now Chris goes the other way: from a proposed regularity back to what a theory must be like to produce it.

Chris's conjecture

- In DOA, practice strengthens problem–answer associations; retrieval should succeed more often, and overt strategies fade.
- Chris proposes something stronger:

  Once the correct answer is the strongest association, strengthening it further never raises the probability of resorting to a backup strategy.

- Plausible, but an empirical claim about what *never* happens: hard to establish directly.
- **Robin**: first ask whether it follows from DOA.

Chris already knows that formalization exposed an ambiguity in one of the chapter's claims (next slide).

Chris's conjecture

- In DOA, practice strengthens problem–answer associations; retrieval should succeed more often, and overt strategies fade.
- Chris proposes something stronger:

  Once the correct answer is the strongest association, strengthening it further never raises the probability of resorting to a backup strategy.

- Plausible, but an empirical claim about what *never* happens: hard to establish directly.
- **Robin**: first ask whether it follows from DOA.

Chris already knows that formalization exposed an ambiguity in one of the chapter's claims (next slide).

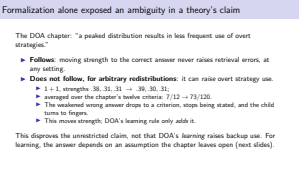
Chris's second question is about practice and strategy choice. In the distributions-of-associations model, practice strengthens the associations between problems and answers, so retrieval should succeed more often and children should rely less on overt strategies like counting. Chris proposes something stronger: once the correct answer is the strongest association, strengthening it further never raises the probability of falling back on a backup strategy. That's plausible, but it's a claim about what never happens, which is hard to establish directly from data. Robin suggests a first step: ask whether it follows from the model at all. And Chris has reason to be careful, because formalization has already exposed an ambiguity in one of the chapter's own claims.

Formalization alone exposed an ambiguity in a theory's claim

The DOA chapter: "a peaked distribution results in less frequent use of overt strategies."

- **Follows**: moving strength to the correct answer never raises retrieval errors, at any setting.
- **Does not follow, for arbitrary redistributions**: it can *raise* overt strategy use.
  - $1 + 1$, strengths $.38, .31, .31 \rightarrow .39, .30, .31$;
  - averaged over the chapter's twelve criteria: $7/12 \rightarrow 73/120$.
  - The weakened wrong answer drops to a criterion, stops being stated, and the child turns to fingers.
  - This *moves* strength; DOA's learning rule only *adds* it.

This disproves the unrestricted claim, not that DOA's *learning* raises backup use. For learning, the answer depends on an assumption the chapter leaves open (next slides).



Here's that earlier result. Formalization alone, with no composition at all, exposed an ambiguity in one of the theories, and it's one of my own. The chapter says that a peaked distribution results in less frequent use of overt strategies. Half of that follows from the model: moving strength to the correct answer never raises retrieval errors, at any setting. Half doesn't, at least for arbitrary changes to the distribution. Peaking can raise overt strategy use. Take one plus one with strengths point three eight, point three one, point three one, and move one hundredth from a wrong answer to the correct one. Averaged over the chapter's own twelve criteria, overt strategy use goes up, from seven twelfths to seventy-three over one twenty. The weakened wrong answer drops to one of the criteria, stops being stated, and the child turns to fingers. So, read as a claim about any change to the distribution, it needs a condition the chapter never stated. But notice one thing about this counterexample, because it matters: it moves strength from a wrong answer to the correct one. That's not what learning does in this model; learning adds strength without taking any from competitors. So the counterexample disproves the unrestricted claim, but it doesn't show that the model's learning raises backup use. Whether learning can do

Robin spots an ambiguity

An answer is drawn in proportion to its strength, and stated only if it **exceeds a criterion** c. Which strength?

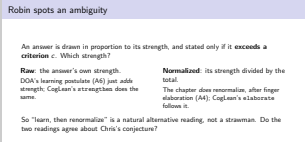
Raw: the answer's own strength.

DOA's learning postulate (A6) just *adds* strength; CogLean's `strengthen` does the same.

Normalized: its strength divided by the total.

The chapter *does* renormalize, after finger elaboration (A4); CogLean's `elaborate` follows it.

So "learn, then renormalize" is a natural alternative reading, not a strawman. Do the two readings agree about Chris's conjecture?



Robin then notices an ambiguity. In the model, an answer is drawn with probability proportional to its strength, and stated only if its strength exceeds a confidence criterion. But which strength: the raw strength, or the strength normalized by the total over all answers? This isn't a pedantic question. The chapter's learning postulate simply adds strength, and the library's learning rule does the same. But the chapter does renormalize in another place, after finger elaboration, and the library follows it there. So learning followed by renormalization is a perfectly natural alternative reading. The question is whether the two readings agree about Chris's conjecture.

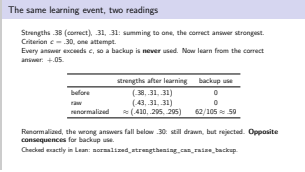
The same learning event, two readings

Strengths .38 (correct), .31, .31: summing to one, the correct answer strongest.
Criterion $c = .30$, one attempt.
Every answer exceeds c, so a backup is **never** used. Now learn from the correct answer: $+.05$.

	strengths after learning	backup use
before	$(.38, .31, .31)$	0
raw	$(.43, .31, .31)$	0
renormalized	$\approx (.410, .295, .295)$	$62/105 \approx .59$

Renormalized, the wrong answers fall below .30: still drawn, but rejected. **Opposite consequences** for backup use.

Checked exactly in Lean: `normalized_strengthening_can_raise_backup`.


The same learning event, two readings

Strengths .38 (correct), .31, .31: summing to one, the correct answer strongest.
Criterion $c = .30$, one attempt.
Every answer exceeds c, so a backup is **never** used. Now learn from the correct answer: $+.05$.

	strengths after learning	backup use
before	(.38, .31, .31)	0
raw	(.43, .31, .31)	0
renormalized	$\approx$ (.410, .295, .295)	$62/105 \approx .59$

Renormalized, the wrong answers fall below .30: still drawn, but rejected. **Opposite consequences** for backup use.
Checked exactly in Lean: `normalized_strengthening_can_raise_backup`.

They try a small example, the same one as the peaking result: the correct answer at point three eight, two wrong answers at point three one, summing to one, so the correct answer is the strongest. The criterion is point three zero and the child gets one retrieval attempt. Every answer exceeds the criterion, under either reading, so the child never uses a backup. Now the child states the correct answer and learns from it, adding point zero five. Under the raw reading the strengths are point four three, point three one, point three one: all still above the criterion, and backup use stays at zero. Under the renormalized reading, divide by the new total, one point zero five: about point four one, and two point two nine fives. Now only the correct answer clears the criterion. The wrong answers can still be drawn, but they're rejected, and with one attempt backup use jumps from zero to sixty-two over one oh five, about point five nine. Same learning event, opposite consequences. Lean checks these exact values. Chris finds the renormalized result surprising: why should strengthening an already dominant correct answer make a child more likely to abandon retrieval? Not impossible, but now there's a concrete theoretical question.

Robin overreaches

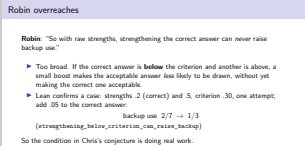
Robin: "So with raw strengths, strengthening the correct answer can *never* raise backup use."

- Too broad. If the correct answer is **below** the criterion and another is above, a small boost makes the acceptable answer *less* likely to be drawn, without yet making the correct one acceptable.
- Lean confirms a case: strengths .2 (correct) and .5, criterion .30, one attempt; add .05 to the correct answer:

$$\text{backup use } \ 2/7 \ \to \ 1/3$$

(`strengthening_below_criterion_can_raise_backup`)

So the condition in Chris's conjecture is doing real work.



Chris asks whether the raw reading guarantees the conjecture, and Robin answers too quickly: with raw strengths, strengthening the correct answer can never raise backup use. That's too broad. Suppose the correct answer is below the criterion and some other answer is above it. A small boost to the correct answer makes the acceptable answer less likely to be drawn, because it's now a smaller share of the total, without yet lifting the correct answer over the criterion. Backup use goes up. Lean confirms a concrete case: strengths point two for the correct answer and point five for another, criterion point three, one attempt. Add point zero five to the correct answer, and backup use rises from two sevenths to one third. So the condition in Chris's conjecture, that the correct answer is already the strongest, is doing real work.

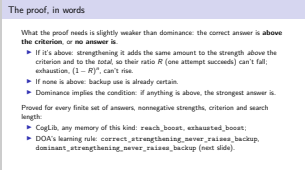
The proof, in words

What the proof needs is slightly weaker than dominance: the correct answer is **above the criterion**, or **no answer is**.

- If it's above: strengthening it adds the same amount to the strength *above* the criterion and to the *total*, so their ratio R (one attempt succeeds) can't fall; exhaustion, $(1 - R)^n$, can't rise.
- If none is above: backup use is already certain.
- Dominance implies the condition: if anything is above, the strongest answer is.

Proved for every finite set of answers, nonnegative strengths, criterion and search length:

- CogLib, any memory of this kind: `reach_boost`, `exhausted_boost`;
- DOA's learning rule: `correct_strengthening_never_raises_backup`, `dominant_strengthening_never_raises_backup` (next slide).

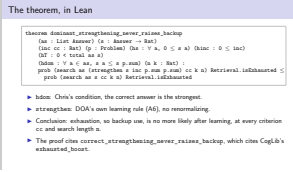


Here's the proof, in words, before the Lean. What it actually needs is a little weaker than dominance: the correct answer is already above the criterion, or no answer is. If it's above, strengthening it adds the same amount to the strength above the criterion and to the total strength. So their ratio, which is the chance that one attempt succeeds, can't go down, and the chance of exhausting retrieval, one minus that ratio to the n, can't go up. If no answer is above the criterion, backup use is already certain, so it can't rise. And dominance implies the condition: if anything clears the criterion, the strongest answer does. This is proved for every finite set of answers, every nonnegative strengths, every criterion and every search length. In the library it's stated generally, for any memory of this kind; and for the strategy-choice model it's stated for its own learning rule. The next slide shows that version.

The theorem, in Lean

```
theorem dominant_strengthening_never_raises_backup
    (as : List Answer) (s : Answer → Rat)
    (inc cc : Rat) (p : Problem) (hs : ∀ a, 0 ≤ s a) (hinc : 0 ≤ inc)
    (hT : 0 < total as s)
    (hdom : ∀ a ∈ as, s a ≤ s p.sum) (n k : Nat) :
    prob (search as (strengthen s inc p.sum p.sum) cc k n) Retrieval.isExhausted ≤
      prob (search as s cc k n) Retrieval.isExhausted
```

- `hdom`: Chris's condition, the correct answer is the strongest.
- `strengthen`: DOA's own learning rule (A6), no renormalizing.
- Conclusion: exhaustion, so backup use, is no more likely after learning, at every criterion `cc` and search length `n`.
- The proof cites `correct_strengthening_never_raises_backup`, which cites CogLib's `exhausted_boost`.

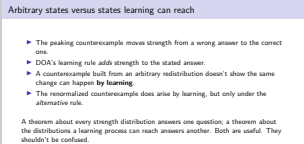


Here's the statement, verbatim. Read the hypotheses: a list of answers and their strengths; an increment and a criterion; nonnegative strengths and increment; a positive total; and h dom, Chris's condition, that every answer is at most as strong as the correct one. The conclusion compares the probability that retrieval is exhausted, which is when the backup is used, after learning with the model's own learning rule, against before, at every criterion and search length. The proof is two steps: it establishes the weaker condition from dominance and cites the general theorem for this learning rule, which in turn cites the library's theorem for any memory of this kind. That's the library working the way it should: Chris's result is a corollary of something general, and the general part is now available to the next person.

Arbitrary states versus states learning can reach

- The peaking counterexample *moves* strength from a wrong answer to the correct one.
- DOA's learning rule *adds* strength to the stated answer.
- A counterexample built from an arbitrary redistribution doesn't show the same change can happen **by learning**.
- The renormalized counterexample does arise by learning, but only under the *alternative* rule.

A theorem about every strength distribution answers one question; a theorem about the distributions a learning process can reach answers another. Both are useful. They shouldn't be confused.



Chris also notices a difference from the peaking result. That counterexample moved strength from a wrong answer to the correct one. But the model's learning rule doesn't move strength; it adds strength to whatever answer was stated. So a counterexample built from an arbitrary redistribution doesn't show that the same change can actually happen through learning. The renormalized counterexample does arise by learning, but only under the alternative rule. The general lesson: a theorem about every possible strength distribution answers one question, and a theorem about the distributions a learning process can actually reach answers another. Both are useful, and it's easy to confuse them.

What has Chris learned?

- The conjecture has become a way to **tell theoretical commitments apart**.
- If backup use really never rises as a dominant correct answer grows, then renormalized DOA learning, unrestricted, is inadequate.
- Raw strengths are *one* way to satisfy it, not the only one; the renormalized model might be rescued by restricting parameters or reachable states.
- Chris hasn't shown that children use raw strengths. Chris has found which assumptions produce, or fail to guarantee, a proposed regularity.

Chris poses · Robin generates alternatives · formalization exposes the ambiguity · Lean settles what follows · evidence decides

What has Chris learned?

- The conjecture has become a way to **tell theoretical commitments apart**.
- If backup use really never rises as a dominant correct answer grows, then renormalized DOA learning, unrestricted, is inadequate.
- Raw strengths are one way to satisfy it, not the only one; the renormalized model might be rescued by restricting parameters or reachable states.
- Chris hasn't shown that children use raw strengths. Chris has found which assumptions produce, or fail to guarantee, a proposed regularity.

Chris poses · Robin generates alternatives · formalization exposes the ambiguity · Lean settles what follows · evidence decides

So what has Chris learned? The conjecture has turned into a way of telling theoretical commitments apart. If it's true that backup use never rises as an already dominant correct answer grows stronger, then the renormalized reading of the model's learning, without further restrictions, is inadequate. Raw strengths are one way to satisfy the regularity, but not the only one: another mechanism might too, and the renormalized model might be rescued by restricting its parameters or the states it can reach. Chris hasn't proved that children use raw strengths. What Chris has done is find out which assumptions produce the regularity and which fail to guarantee it. That's reasoning backward, from a phenomenon toward possible mechanisms. Chris posed the question, Robin generated alternatives and got one wrong, formalization exposed an ambiguity, Lean settled what follows from what, and evidence will have to decide which premises describe children. And all of it is now in the library, for the next person to build on.

Part 3

When theories and phenomena aren't exact

The power law of practice, unbundled

Part three. Everything so far was exact. What if the theory has random or unspecified parts, or the phenomenon is only roughly characterized, like a learning curve that looks like a power law?

Imprecise mechanisms

1. **Probabilistic, with known probabilities**: still precise. The outcome distribution is a mathematical object; CogLib computes and reasons about it **exactly**. (Everything so far.)
2. **Monte Carlo**: the distribution is defined, but expensive. Sampling makes the consequences harder to calculate, not the theory imprecise. Results are **estimates**, checked against exact values where both exist.
3. **Incompletely specified**: a heuristic, a table, an unknown process. It becomes a **parameter**, and what it must satisfy becomes a **condition**. (Sharp associations.)

A model can mix deduced and estimated results, if it says which is which.

Imprecise mechanisms

1. **Probabilistic, with known probabilities**: still precise. The outcome distribution is a mathematical object; CogLib computes and reasons about it **exactly**. (Everything so far.)
2. **Monte Carlo**: the distribution is defined, but expensive. Sampling makes the consequences harder to calculate, not the theory imprecise. Results are **estimates**, checked against exact values where both exist.
3. **Incompletely specified**: a heuristic, a table, an unknown process. It becomes a **parameter**, and what it must satisfy becomes a **condition**. (Sharp associations.)

A model can mix deduced and estimated results, if it says which is which.

First, squishiness at the bottom, in the mechanisms. Three cases. A mechanism that makes random choices with known probabilities is still precise: its distribution of outcomes is a mathematical object, and the library computes and reasons about it exactly. Everything I've shown so far is of that kind. Second, Monte Carlo: the distribution is defined, but too expensive to compute, so we estimate it by running the mechanism many times. That doesn't make the theory imprecise; it makes its consequences harder to calculate, and the results are statistical estimates, checked against exact values where we have both. Third, a part of the model that is genuinely unspecified: a heuristic, an empirical table, a process whose distribution we don't know. Deduction still works, with that part as a parameter and what it must satisfy as a condition. Sharp associations were exactly that. So a model can mix deduced and estimated results, as long as it says which is which.

Imprecise phenomena: the power law, unbundled

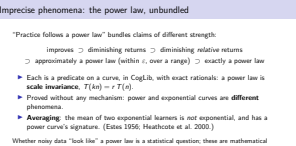
"Practice follows a power law" bundles claims of different strength:

improves $\supset$ diminishing returns $\supset$ diminishing *relative* returns

$\supset$ approximately a power law (within ε, over a range) $\supset$ exactly a power law

- Each is a predicate on a curve, in CogLib, with exact rationals: a power law is **scale invariance**, $T(kn) = r\,T(n)$.
- Proved without any mechanism: power and exponential curves are **different** phenomena.
- **Averaging**: the mean of two exponential learners is *not* exponential, and has a power curve's signature. (Estes 1956; Heathcote et al. 2000.)

Whether noisy data "look like" a power law is a statistical question; these are mathematical ones.



Now squishiness at the top, in the phenomenon. Saying that practice follows a power law bundles several claims of different strength: performance improves; the gains diminish; the proportional gains diminish; the curve is within some tolerance of a power law over some range of trials; and, strongest, it is a power law. The library states each one separately, as a predicate on a curve. Since n to a fractional power is usually irrational, a power law is stated as scale invariance: multiplying practice by k multiplies time by a constant. Some relations follow with no mechanism at all. Power and exponential curves are different phenomena: no positive curve is both. And here's one I like: average two learners who are each exactly exponential, with different rates, and the average isn't exponential; it has the signature of a power curve. That's Estes's old warning about group curves, and it's at the center of the debate about power versus exponential learning, now as a theorem. One caution: whether noisy data look like a power law is a statistical question. These are mathematical ones.

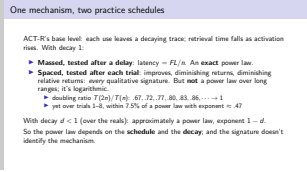
One mechanism, two practice schedules

ACT-R's base level: each use leaves a decaying trace; retrieval time falls as activation rises. With decay 1:

- **Massed, tested after a delay**: latency $= FL/n$. An **exact** power law.
- **Spaced, tested after each trial**: improves, diminishing returns, diminishing relative returns: *every* qualitative signature. But **not** a power law over long ranges; it's logarithmic.
  - doubling ratio $T(2n)/T(n)$: $.67, .72, .77, .80, .83, .86, \cdots \to 1$
  - yet over trials 1–8, within 7.5% of a power law with exponent $\approx .47$

With decay $d < 1$ (over the reals): approximately a power law, exponent $1 - d$.

So the power law depends on the **schedule** and the **decay**; and the signature doesn't identify the mechanism.



Now a mechanism. In ACT-R's base-level learning, each use of a chunk leaves a trace that decays, and retrieval time falls as activation rises. With decay one, which keeps everything rational, the same mechanism gives two different curves. Practice all at once and test after a delay, and latency is exactly F L over n: an exact power law. Practice spaced out, one use per trial, tested after each, and the curve improves, with diminishing returns and diminishing relative returns: every qualitative signature of a power law. But it isn't one. The ratio from doubling practice creeps up toward one, point six seven, point seven two, point seven seven, and on, so no fixed power law fits it over a long range; it's logarithmic. Yet over the first eight trials it's within seven and a half percent of one. So the power law depends on the schedule, a condition, and the signature doesn't identify the mechanism: averaged exponentials have it too. With ACT-R's usual decay of a half, over the reals, spaced practice is approximately a power law with exponent one minus the decay: Anderson's account, checked. The general point: CogLean doesn't need theories or phenomena to be exact. It needs the relations we assert between them to be explicit.

Part 4

What this shows, and what it doesn't

Kinds of results, where it sits, limits

Finally, what all this shows and doesn't: how results are labeled, where CogLean sits, and its limits.

What each result rests on

Kind	Examples
Library facts	a sample is a probability distribution; two evaluators agree
Compositional (every setting)	after certain recognition, backup use depends only on what's recognized
Conditional	confident errors; ACT-R's practice curves, given a schedule
Class-level	any memory with three laws; any non-interfering learner reaches criterion
Relations between phenomena	power $\neq$ exponential; averaged exponentials aren't exponential
At chosen settings	45/46; 225/391; within 7.5% over trials 1–8

None of these is yet an empirical finding about people. The numbers are witnesses, not explanations, and not fits.

What each result rests on

Kind	Examples
Library facts	a sample is a probability distribution; two evaluators agree
Compositional (every setting)	after certain recognition, backup use depends only on what's recognized
Conditional	confident errors; ACT-R's practice curves, given a schedule
Class-level	any memory with three laws; any non-interfering learner reaches criterion
Relations between phenomena	power $\neq$ exponential; averaged exponentials aren't exponential
At chosen settings	45/46, 225/391; within 7.5% over trials 1–8

None of these is yet an empirical finding about people. The numbers are witnesses, not explanations, and not fits.

For credibility, it's essential to say what each result rests on. Some are library facts: needed, but not psychology. Some are compositional deductions that hold at every setting. Some are conditional, like confident errors, or ACT-R's practice curves given a schedule, and the conditions are where the science is. Some hold for whole classes of mechanisms that satisfy stated laws. Some relate phenomena to each other, with no mechanism at all. And some are just numbers at chosen settings. A number at one setting is a witness that the conditions can be met, not an explanation; the explanation is the theorem. None of these is yet an empirical finding about people, and none is a fit to data.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Model verification and theoretical deduction

- **Simulation** samples executions.
- **Probabilistic model checking** (PRISM; Soar to PRISM, Ganeriwala et al. 2026) analyzes a specified state-transition model exhaustively; it is formal verification, and can treat parameters symbolically.
- **Deductive proof** (CogLean) establishes results for whole classes of models satisfying theoretical assumptions.

The contrast is *model verification* versus *theoretical deduction*. Complementary, not competitors.

CogLean's claim is not formal methods (see the prior work), but the combination: a shared, cumulative library with reusable proofs, cross-theory composition, and deductive discovery.

Where does this sit? There's a lot of prior formal work on cognitive architectures: a specification of Soar in Z, the Sceptic specification language, formal semantics and model checking for ACT-R, and recently Soar models compiled for the PRISM probabilistic model checker. It helps to separate three approaches. Simulation samples executions. Probabilistic model checking analyzes a specified model exhaustively; that's real formal verification, not simulation, and it can treat parameters symbolically. Deductive proof establishes results for whole classes of models that satisfy theoretical assumptions. So the contrast isn't simulation versus verification; it's model verification versus theoretical deduction, and they're complementary. CogLean doesn't introduce formal methods to cognitive science. What's new is the combination: a shared, cumulative library with reusable proofs, composition across theories, and deductive discovery.

Relationship to cognitive architectures

- **Complementary**, not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- Architectures accumulate shared mechanisms and models; CogLean tries to accumulate **proved results** about mechanisms, including over classes of them. An architecture could carry such results itself.
- Example: confident errors hold for every memory with three laws (EPAM has them; ACT-R's memory has none). The laws are sufficient, not shown necessary; no general equivalence or divergence of architectures is claimed.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present library composes locally, for particular tasks, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

Relationship to cognitive architectures

- **Complementary**, not an alternative to Soar, ACT-R, or the Common Model of Cognition.
- Architectures accumulate shared mechanisms and models; CogLean tries to accumulate **proved results** about mechanisms, including over classes of them. An architecture could carry such results itself.
- Example: confident errors hold for every memory with three laws (EPAM has them; ACT-R's memory has none). The laws are sufficient, not shown necessary; no general equivalence or divergence of architectures is claimed.
- A possible **formal foundation** for expressing, composing and reasoning about their mechanisms and architectural commitments.
- The present library composes locally, for particular tasks, without committing to a general architecture.
- Longer term: formalize **architectural constraints themselves**, so their implications can be proved and architectures compared in one framework.

And to be clear about cognitive architectures. CogLean is complementary to Soar, ACT-R and the Common Model of Cognition, not an alternative to them. Architectures accumulate theory as shared mechanisms in which models are built. CogLean tries to accumulate proved results about those mechanisms, including results over whole classes of them, and nothing stops an architecture from carrying such results itself. The confident-errors theorem is the example: it holds for every memory that satisfies three laws, which EPAM's memory has and ACT-R's has none of. That's also the limit of what's shown: the laws are sufficient, not shown to be necessary, and nothing here establishes general equivalence or divergence between architectures. It's a possible formal foundation for expressing, composing and reasoning about their mechanisms and architectural commitments. The present library composes mechanisms locally, for particular tasks, without committing to any general architecture. The longer-term aim is to formalize architectural constraints themselves, so that their implications can be proved and alternative architectures compared within one shared framework.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

What a proof shows, and what it doesn't

- Lean guarantees **validity**: conclusions follow from premises.
- Not **empirical adequacy**: whether the premises describe people.
- Not **explanation**: whether the premises already contain the phenomenon.
- Example: a serial-position result whose mechanism was a bowed schedule. Valid, nearly empty. Now derived from an attention rule, which also gives the isolation effect.
- An explicit audit sorts results as definitional, close, derived, compositional.

A word on what proofs do and don't show. Lean guarantees validity. It doesn't guarantee that the premises describe people, and it doesn't guarantee explanation, because the premises might already contain the phenomenon. We had exactly that case: a serial-position result whose mechanism was a bowed schedule, so proving a bowed curve was valid and nearly empty. It's now derived from an attention rule over anchors, which also gives the isolation effect, a second consequence of the same mechanism. The library keeps an explicit audit that sorts its results as definitional, close, derived or compositional. That's a judgment, written down in the open.

What CogLean is, and isn't

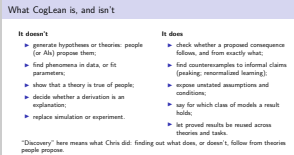
It doesn't

- generate hypotheses or theories: people (or AIs) propose them;
- find phenomena in data, or fit parameters;
- show that a theory is true of people;
- decide whether a derivation is an explanation;
- replace simulation or experiment.

It does

- check whether a proposed consequence follows, and from exactly what;
- find counterexamples to informal claims (peaking; renormalized learning);
- expose unstated assumptions and conditions;
- say for which class of models a result holds;
- let proved results be reused across theories and tasks.

"Discovery" here means what Chris did: finding out what does, or doesn't, follow from theories people propose.



So, to be clear about what this is and isn't. CogLean doesn't generate hypotheses or theories; people, or AIs, propose them. It doesn't find phenomena in data or fit parameters. It doesn't show that a theory is true of people, and it can't decide whether a derivation is an explanation; that's a scientific judgment. And it doesn't replace simulation or experiment. What it does is check whether a proposed consequence actually follows, and from exactly which premises. It finds counterexamples to informal claims, like the peaking claim. It exposes assumptions and conditions that theories leave unstated. It says for which class of models a result holds. And it lets proved results be reused across theories and tasks. When I say discovery, that's what I mean: finding out what does, or doesn't, follow from theories that people propose.

Directions

- **Composing theories** (companion talk): EPAM + strategy choice + a problem solver → an executable model of three-digit addition, with consequences proved largely from the parts' theorems.
- **A second domain** (companion): concept formation by view application, on the same memory interface and laws.
- **Classes of mechanisms**: more families and laws; graded laws for graded phenomena.
- **Quantitative phenomena**: other theories' learning curves on the same predicates; statistical versions, for noisy data.
- **Several explanations**: what primes; when the explanations diverge.

Directions

- **Composing theories** (companion talk): EPAM + strategy choice + a problem solver → an executable model of three-digit addition, with consequences proved largely from the parts' theorems.
- **A second domain** (companion): concept formation by view application, on the same memory interface and laws.
- **Classes of mechanisms**: more families and laws; graded laws for graded phenomena.
- **Quantitative phenomena**: other theories' learning curves on the same predicates; statistical versions, for noisy data.
- **Several explanations**: what primes; when the explanations diverge.

Where this goes next. The companion talk composes EPAM, strategy choice and a problem solver into an executable model of three-digit addition, with its consequences proved largely by reusing the parts' theorems. Another companion takes the library to concept formation, by view application, on the same memory interface and laws. The theory matrix should grow, in families and laws, including graded laws for graded phenomena. Other theories' learning curves should be stated on the same predicates, so rival mechanisms can be compared quantitatively, and we need statistical versions for noisy data. And for confident errors, we need an account of what does the priming, and of exactly when the two explanations come apart.

Chris and Robin: from reasoning to model construction

A future possibility, not a present accomplishment.

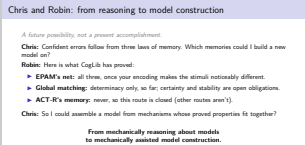
Chris: Confident errors follow from three laws of memory. Which memories could I build a new model on?

Robin: Here is what CogLib has proved:

- **EPAM's net:** all three, once your encoding makes the stimuli noticeably different.
- **Global matching:** determinacy only, so far; certainty and stability are open obligations.
- **ACT-R's memory:** never, so this route is closed (other routes aren't).

Chris: So I could assemble a model from mechanisms whose proved properties fit together?

From mechanically reasoning about models
to mechanically assisted model construction.



A last turn of the story, and a possibility, not something built. Once a theorem holds for every memory that satisfies stated laws, you can turn the question around: which mechanisms in the library meet the requirements of a result you want your model to have? Chris asks which memories could support confident errors, and Robin can answer from the theory matrix, where we did this search by hand. EPAM's net has all three laws, but certainty needs noticeably different stimuli, a property of the encoding that the rest of Chris's model must guarantee: an obligation handed to another component. Global matching has determinacy, but certainty and stability are still open. ACT-R's memory provably never has them, so this route is closed for it, though not every route to confident errors. A failed obligation doesn't show that no suitable model exists, and the laws need hold only in states the model can reach. With enough mechanisms, laws and theorems, the library could guide the construction of models, not just analyze them. Lean gives us proof checking, automation and instance search; the organization and search machinery don't exist yet. None of this invents theories or makes a model psychologically plausible: the matrix judges that people lack certainty, so the one memory that fits isn't a good model of people. The direction:

Limitations

- No contact with data yet; plausibility is for evidence and judgment.
- Key conditions are assumed, not derived (sharp associations); strengths are chosen, not learned.
- Exact rationals: ACT-R mostly at decay 1 and temperature 1 (reals only for learning curves), and without retrieval as use.
- Scalability: exact finite distributions; continuous and neural models need other techniques.
- Nonnegative weights not enforced by the type, nor proved library-wide.
- A small library, of theories chosen partly because they could be formalized.

Limitations

- No contact with data yet; plausibility is for evidence and judgment.
- Key conditions are assumed, not derived (sharp associations); strengths are chosen, not learned.
- Exact rationals: ACT-R mostly at decay 1 and temperature 1 (reals only for learning curves), and without retrieval as use.
- Scalability: exact finite distributions; continuous and neural models need other techniques.
- Nonnegative weights not enforced by the type, nor proved library-wide.
- A small library, of theories chosen partly because they could be formalized.

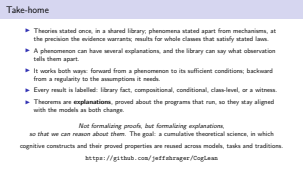
The limitations are real. There's no contact with data yet, and psychological plausibility is a matter for evidence and judgment, not proof. Key conditions, like sharp associations, are assumed rather than derived, and the strengths in the examples are chosen, not learned. Exact rationals restrict some theories to special cases: ACT-R appears mostly with decay one and temperature one, with real numbers so far only for learning curves, and without retrieval strengthening what it retrieves. Exact finite distributions raise real questions of scale; continuous and neural models will need other techniques. The distribution type doesn't enforce nonnegative weights, and that's not yet proved across the library. And the library is small, built from theories chosen partly because they could be formalized.

Take-home

- Theories stated once, in a shared library; phenomena stated apart from mechanisms, at the precision the evidence warrants; results for whole classes that satisfy stated laws.
- A phenomenon can have several explanations, and the library can say what observation tells them apart.
- It works both ways: forward from a phenomenon to its sufficient conditions; backward from a regularity to the assumptions it needs.
- Every result is labelled: library fact, compositional, conditional, class-level, or a witness.
- Theorems are **explanations**, proved about the programs that run, so they stay aligned with the models as both change.

Not formalizing proofs, but formalizing explanations,
so that we can reason about them. The goal: a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and traditions.

`https://github.com/jeffshrager/CogLean`



To sum up. Cognitive theories can be stated once, in a shared formal library, with their consequences proved rather than simulated. Phenomena are stated apart from any mechanism, at whatever precision the evidence warrants, and results can hold for whole classes of mechanisms that satisfy stated laws. A phenomenon can have more than one explanation, and the library can say which observation would tell them apart. It works in both directions, as Chris's two questions showed: forward from a phenomenon to conditions sufficient for it, and backward from a proposed regularity to the assumptions it needs. And every result is labelled for what it rests on. The goal is a cumulative theoretical science, in which cognitive constructs and their proved properties are reused across models, tasks and research traditions, the way mathematical results are reused in Mathlib. This doesn't establish that. It shows the approach can produce results of the right kind. CogLean isn't primarily about formalizing proofs. It's about formalizing explanations so that we can reason about them, and using Lean to keep those explanations rigorously aligned with the computational models they explain.